

The reason why I took screenshots of my own doc was because the document was corrupted; everything was overlapping each other.

22AIE303

DBMS: PLSQL - LAB1

Name: Sravan Sudeep

RollNo: AM.SC.U4AIE23153

1. Write a function that finds the sum of two ints.

```
-- 1. Do all the questions discussed in the class
create or replace function sum2(a1 int, b int)
returns int as
$$
Declare
    c int;
begin
    c:=a1+b;
    return c;
End;
$$
language 'plpgsql';

select sum2(5,10)
```

	sum2 integer 
1	15

2. write a function that accepts a int value and returns even or odd

```

1  create or replace function odd(a1 int)
2  returns varchar as
3  $$
4  begin
5      if a1%2=0 then
6          return 'even';
7      else
8          return 'odd';
9      end if;
10 End;
11 $$
12 language 'plpgsql';
13
14 select odd(5)

```

	odd
	character varying 
1	odd

Create relations:

```

create table Emp(
    eno int primary key,
    ename varchar(20),
    sal numeric(10,2),
    dno int
);

create table Dept(
    dept_no int primary key,
    dname varchar(20),
    loc varchar(30)
);

```

1. Write a function that accepts an emp name and returns the sal of the emp

```
1 CREATE OR REPLACE FUNCTION find_sal(empno emp.eno%type)
2 RETURNS emp.sal%type AS
3 $$
4 DECLARE
5     salary emp.sal%type;
6 BEGIN
7     SELECT sal INTO salary FROM Emp WHERE eno = empno;
8     RETURN salary;
9 END;
10 $$
11 LANGUAGE plpgsql;
12
13 SELECT find_sal(101);
```

Out:

	find_sal numeric 
1	45000.00

2. Write a function that accepts dno and finds the no of employees

```

1  create or replace function emp_count(deptno emp.dno%type)
2      returns int as
3      $$
4          Declare
5              num int;
6      Begin
7          select count(eno) into num from Emp where dno=deptno;
8          return num;
9      End;
10     $$
11  Language 'plpgsql';
12
13  SELECT emp_count(10)

```

Out:

emp_count	
integer	
1	2

- Write a function that gives the emp 10% hike in salary if avg salary of the corresponding department is greater than employee's current salary.

```

1 CREATE OR REPLACE FUNCTION promo(empno emp.eno%TYPE)
2 RETURNS void AS
3 $$
4 DECLARE
5     salary emp.sal%TYPE;
6     avg_sal numeric;
7 BEGIN
8     SELECT AVG(sal) INTO avg_sal FROM Emp
9     WHERE dno = (SELECT dno FROM Emp WHERE eno = empno);
10
11     SELECT sal INTO salary FROM Emp WHERE eno = empno;
12
13 IF salary < avg_sal THEN
14     salary := salary + (salary * 0.10);
15     UPDATE Emp SET sal = salary WHERE eno = empno;
16 END IF;
17 END;
18 $$
19 LANGUAGE plpgsql;
20
21 SELECT promo(101)
22

```

Out: (not returning anything)

	promo
	void
1	

- Write a function that accepts an employee name and returns the dep name of that employee

```

1 create or replace function emp_dep(ename emp.ename%type)
2 returns Dept.dname%type as
3 $$
4     Declare
5     dep Dept.dname%type;
6     Begin
7         select dname into dep from Dept where dept_no =(select dno from Emp where ename=ename);
8         return dep;
9     End;
10 $$
11 Language 'plpgsql';
12
13 SELECT emp_dep('Ravi');

```

Out:

	emp_dep character varying 
1	HR

2. Consider the following schema .
 Emp(eno, ename, sal)
 EMP_PROJ(eno, pno, prjt_hrs).

- Write a function ProjectLoad that returns the total project working hours for the given eno.
-
- Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```

create table emp_proj(
eno int references Emp(eno),
pno int,
prjt_hrs numeric(6,2),
primary key (eno,pno)
);

```

a)

```
1 create or replace function ProjectLoad(empno emp_proj.eno%type)
2   returns emp_proj.prjt_hrs%type as
3   $$
4     Declare
5       total emp_proj.prjt_hrs%type;
6     Begin
7       select sum(prjt_hrs) into total from emp_proj group by eno having eno=empno;
8       return total;
9     End;
10  $$
11  Language 'plpgsql';
12
13 select ProjectLoad(102)
```

Out:

	projectload numeric
1	35.75

b)

```
create or replace function sal_update(empno emp.eno%type)
returns void as
$$
  Declare
    salary emp.sal%type;
    avg int;
  Begin
    select avg(sal) into avg from Emp where dno (select dno from Emp where eno=empno);
    select sal into salary from Emp where eno=empno;
    if salary<avg then
      salary:=salary+(avg_sal-salary);
      update Emp set sal=salary where eno=empno;
    end if;
  End;
$$
Language 'plpgsql';
```

Out:

The salary was updated -> output is shown as empty due to no return statement

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

Create Relation:

```
create table Item(  
ino int primary key,  
iname varchar(10),  
unit_price numeric(8,2)  
);  
  
create table Transaction(  
tr_no int,  
ino int references Item(ino),  
qty int,  
primary key(tr_no,ino)  
);
```

Q1.

```
1  create or replace function item_del(itno item.ino%type)  
2  returns void as  
3  $$  
4      declare  
5          amt int;  
6  begin  
7      select count(tr_no) into amt from transaction where ino = itno;  
8      if amt < 2 then  
9          delete from item where ino = itno;  
10         end if;  
11     end;  
12 $$  
13 language plpgsql;  
14  
15 SELECT item_del(1);
```

Out:

Item was deleted for amt of transcations less than 2

	item_del void 
1	

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no, br_no, cust_no, ac_type, bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

- Write a function that accepts a threshold value and a customer number .
The program updates the c_type based on the threshold value. Ie. If balance > threshold then class A, else class B.
- Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

Create relations:

```
create table Branches(  
  br_no int primary key,  
  br_name varchar(20),  
  loc varchar(50)  
);  
  
create table Customer(  
  cno int primary key,  
  cname varchar(10),  
  c_type varchar(10)  
);  
  
create table Accounts(  
  ac_no int primary key,  
  br_no int references Branches(br_no),  
  cust_no int references Customer(cno),  
  ac_type varchar(15),  
  bal numeric(10,2)  
);
```

1.

```
1 create or replace function update_customer(threshold numeric, custno customer.cno%type)
2 returns text as
3 $$
4     Declare
5         total_bal numeric;
6     Begin
7         select sum(bal) into total_bal from accounts where cust_no = custno;
8         if total_bal > threshold then
9             update customer set c_type = 'A' where cno = custno;
10            return 'Customer upgraded to A';
11        else
12            update customer set c_type = 'B' where cno = custno;
13            return 'Customer downgraded to B';
14        end if;
15    End;
16 $$
17 language plpgsql;
18 |
19 select update_customer(4000, 101);
```

Out:

	update_customer	
	text	
1	Customer upgraded to A	

2.

```
create or replace function closebranch(old_br branches.br_no%type, new_br branches.br_no%type)
returns void as
$$
begin
    update accounts set br_no = new_br where br_no = old_br;

    delete from branches where br_no = old_br;
end
$$
language plpgsql;

select closebranch(2, 1);
```

Out:

Returns void so won't be shown but acnts was moved to different branch for that id.

3.

```
1 create or replace function Safewithdraw(accno accounts.ac_no%type, amount numeric)
2 returns varchar as
3 $$
4     Declare
5         current_bal numeric;
6     Begin
7         select bal into current_bal from accounts where ac_no = accno;
8         if current_bal >= amount then
9             update accounts set bal = bal - amount where ac_no = accno;
10            return 'Withdrawal successful';
11        else
12            return 'Insufficient funds';
13        end if;
14    End;
15 $$
16 Language plpgsql;
17
18 select safewithdraw(1001, 500);
19
```

Out:

	safewithdraw	
	character varying	
1	Withdrawal successful	